UTEC
Universidad Tecnológica

# Patrón Decorator

El **Patrón Decorator** es un patrón de diseño estructural que permite agregar funcionalidades a objetos de manera flexible y dinámica. A diferencia de la herencia, que agrega comportamiento a una clase en su totalidad, el Decorator agrega o modifica funcionalidades en un solo objeto de manera selectiva.

**¿Cuándo usar el Patrón Decorator?**

- Cuando quieres agregar funcionalidades a un objeto individual, sin afectar el comportamiento de otros objetos de la misma clase.
- Cuando necesitas añadir responsabilidades adicionales a los objetos de forma flexible y reversible.
- Cuando la herencia no es viable debido a la necesidad de múltiples combinaciones de responsabilidades.

**Estructura del Patrón Decorator**

1. **Componente (Component):** Es una interfaz o clase abstracta que define las operaciones que pueden ser decoradas.
2. **Componente Concreto (ConcreteComponent):** Es una implementación del Componente, que puede ser decorada.
3. **Decorador (Decorator):** Es una clase abstracta que implementa la interfaz del Componente y contiene una referencia a un objeto del tipo Componente.
4. **Decoradores Concretos (ConcreteDecorators):** Son clases que extienden el Decorador y agregan funcionalidades adicionales al Componente.

**Ejemplo en Java: Decoración de una Ventana**

Supongamos que tenemos un sistema de interfaz de usuario con diferentes tipos de ventanas, y queremos agregar funcionalidades como bordes, barras de desplazamiento, etc., sin modificar las clases base.

**1. Definición del Componente:**

```
interface Ventana {
    void dibujar(); // Método que dibuja la ventana
}
```

UTEC
Universidad Tecnológica

```
class VentanaSimple implements Ventana {
    @Override
    public void dibujar() {
        System.out.println("Dibujar ventana simple");
    }
}
```

**2. Definición del Decorador:**

```
abstract class DecoradorVentana implements Ventana {
    protected Ventana ventanaDecorada;

    public DecoradorVentana(Ventana ventanaDecorada) {
        this.ventanaDecorada = ventanaDecorada;
    }

    @Override
    public void dibujar() {
        ventanaDecorada.dibujar();
    }
}
```

**3. Implementación de Decoradores Concretos:**

```
class BordeDecorador extends DecoradorVentana {
    public BordeDecorador(Ventana ventanaDecorada) {
        super(ventanaDecorada);
    }

    @Override
    public void dibujar() {
        super.dibujar();
        dibujarBorde();
    }

    private void dibujarBorde() {
        System.out.println("Dibujar un borde a la ventana");
```

UTEC
Universidad Tecnológica

```
    }
}

class BarraDesplazamientoDecorador extends DecoradorVentana {
    public BarraDesplazamientoDecorador(Ventana ventanaDecorada) {
        super(ventanaDecorada);
    }

    @Override
    public void dibujar() {
        super.dibujar();
        dibujarBarraDesplazamiento();
    }

    private void dibujarBarraDesplazamiento() {
        System.out.println("Dibujar una barra de desplazamiento");
    }
}
```

**4. Uso del Patrón Decorator:**

```
public class Main {
    public static void main(String[] args) {
        // Crear una ventana simple
        Ventana ventanaSimple = new VentanaSimple();

        // Decorar la ventana con un borde
        Ventana ventanaConBorde = new BordeDecorador(ventanaSimple);
        ventanaConBorde.dibujar();
        System.out.println();

        // Decorar la ventana con un borde y una barra de desplazamiento
        Ventana ventanaConBordeYBarra = new
BarraDesplazamientoDecorador(ventanaConBorde);
        ventanaConBordeYBarra.dibujar();
    }
}
```

UTEC
Universidad Tecnológica

**Salida del Programa:**

```
Dibujar ventana simple

Dibujar un borde a la ventana


Dibujar ventana simple

Dibujar un borde a la ventana

Dibujar una barra de desplazamiento
```

## Ventajas del Patrón Decorator

1. **Flexibilidad:** Permite agregar responsabilidades a objetos individuales sin afectar a otros objetos de la misma clase.
2. **Reutilización de Código:** Puedes combinar decoradores de diferentes maneras para crear combinaciones únicas de funcionalidades.
3. **Responsabilidades Divididas:** Cada decorador tiene una única responsabilidad, lo que facilita la comprensión y el mantenimiento del código.

## Desventajas del Patrón Decorator

1. **Complejidad Incrementada:** Al utilizar múltiples decoradores, el código puede volverse más complejo y difícil de seguir.
2. **Orden de Decoradores:** El orden en que aplicas los decoradores puede afectar el resultado, lo que podría conducir a errores si no se maneja correctamente.

## Conclusión

El Patrón Decorator es ideal cuando necesitas agregar o modificar el comportamiento de los objetos de manera dinámica y flexible. Es una excelente alternativa a la herencia cuando se busca mantener el código más modular y manejable. Sin embargo, como con cualquier patrón de diseño, debe usarse con precaución para evitar complejidades innecesarias en el código.